
1. Historia del Lenguaje de Modelado Unificado (UML)

El **UML (Unified Modeling Language)** no nació de la nada; fue la solución a la "guerra de métodos" que existía en la ingeniería de software a principios de los años 90.

El Caos de los Métodos (Pre-1994)

Antes de UML, los desarrolladores utilizaban decenas de lenguajes de modelado diferentes (OMT, Booch, OOSE). Esto causaba confusión: un símbolo podía significar una "clase" en un método y algo totalmente distinto en otro.

La Unión de "Los Tres Amigos"

La historia de UML se define por tres figuras clave en Rational Software:

1. **Grady Booch:** Creador del método Booch (excelente para el diseño).
2. **James Rumbaugh:** Creador de OMT (Object Modeling Technique), ideal para el análisis.
3. **Ivar Jacobson:** Creador de OOSE (Object-Oriented Software Engineering), quien introdujo los **Casos de Uso**.

En 1996, unieron sus esfuerzos para crear un estándar único. El resultado fue UML 1.0, presentado a la **OMG (Object Management Group)**.

Evolución y Estandarización

- **1997 (UML 1.1):** Adoptado oficialmente por la OMG como estándar de la industria.
- **2005 (UML 2.0):** Una de las actualizaciones más importantes. Se añadieron diagramas de estructura compuesta y se refinaron los diagramas de interacción para manejar sistemas mucho más complejos y distribuidos.

2. Requerimientos de Sistemas

Un requerimiento es una descripción de un servicio que el sistema debe proporcionar o una restricción bajo la cual debe operar. Es el "qué" debe hacer el sistema antes de decidir el "cómo".

El proceso de Ingeniería de Requerimientos:

1. **Elicitación:** Descubrir qué necesitan los stakeholders (entrevistas, talleres).
2. **Análisis:** Detectar conflictos, ambigüedades y omisiones.
3. **Especificación:** Documentar los requerimientos (usualmente en un documento llamado SRS o ERS).
4. **Validación:** Confirmar con el cliente que lo escrito es realmente lo que desea.

3. Requerimientos Funcionales

Los **requerimientos funcionales** definen las acciones específicas que el sistema debe ser capaz de realizar. Describen el comportamiento del software ante entradas particulares.

Características:

- Son medibles y observables.
- Se suelen expresar como: *"El sistema debe [hacer X acción]"*.

Ejemplos comunes:

- "El sistema debe permitir al usuario recuperar su contraseña mediante correo electrónico".
- "El sistema debe generar un reporte de ventas mensual en formato PDF".
- "Al realizar un pago, el sistema debe descontar automáticamente el stock del producto".

Relación con UML: Los requerimientos funcionales se modelan típicamente mediante **Diagramas de Casos de Uso**, donde se visualiza al actor (usuario) y la función que desea realizar.

4. Requerimientos No Funcionales (RNF)

A menudo llamados "atributos de calidad", los **requerimientos no funcionales** no describen *qué* hace el sistema, sino *cómo* lo hace o bajo qué condiciones.

Categorías principales:

Para entenderlos mejor, se suelen clasificar según el modelo **URPS** (Usabilidad, Confiabilidad, Rendimiento y Soporte):

Categoría	Descripción	Ejemplo
Rendimiento	Velocidad y eficiencia del sistema.	"El sistema debe cargar la página principal en menos de 2 segundos".
Seguridad	Protección de datos y acceso.	"Todas las contraseñas deben estar encriptadas con el algoritmo SHA-256".
Usabilidad	Facilidad de uso para el usuario final.	"Un usuario nuevo debe ser capaz de realizar una compra sin entrenamiento previo".
Disponibilidad	Tiempo que el sistema está operativo.	"El sistema debe tener un uptime del 99.9% anual".
Escalabilidad	Capacidad de crecer ante más demanda.	"La base de datos debe soportar hasta 10,000 usuarios concurrentes sin degradar el servicio".

Conclusión: El puente entre Requerimientos y UML

La ingeniería de software moderna utiliza los requerimientos como la base (la necesidad del negocio) y a UML como el lenguaje para comunicar esa necesidad a los programadores. Mientras los **requerimientos funcionales** se convierten en diagramas de secuencia y casos de uso, los **no funcionales** a menudo dictan la arquitectura física que se verá en los diagramas de despliegue.

¿Te gustaría que profundicemos en algún diagrama de UML específico (como el de Clases o Secuencia) para aplicarlo a estos requerimientos?

